

Busy Beans Backend - Complete Project Overview

Generated: December 11, 2025

Project: Busy Beans Coffee Distribution Platform

Version: 1.0.0



Executive Summary

Busy Beans Backend is a comprehensive Node.js/Express API for managing a coffee distribution business. The system handles customer orders, partner/local distributor workflows, inventory management, QuickBooks accounting integration, Stripe payment processing, subscription management, and lead tracking.



Architecture Overview

Technology Stack

Core Framework:

- **Runtime:** Node.js ($\geq 10.0.0$)
- **Framework:** Express.js
- **Database:** MySQL with Sequelize ORM
- **Caching:** Redis (via ioredis)

Key Integrations:

- **Payment Processing:** Stripe (customer payments, partner payouts, subscriptions)
- **Accounting:** QuickBooks Online API (invoice sync, payment tracking)
- **Email:** Nodemailer (transactional emails, quotations, invoices)
- **Notifications:** Firebase Admin (push notifications)
- **File Storage:** Local file system with Sharp for image processing
- **PDF Generation:** Puppeteer

Security & Performance:

- JWT authentication with Redis session management

- bcryptjs for password hashing
- Helmet, CORS, express-rate-limit for API security
- Compression middleware

Server Configuration

- **Entry Point:** bb.js (starts the server)
- **App Configuration:** app.js (Express app setup)
- **Port:** 8011 (default)
- **Host:** 192.168.1.110 (configured for local network)
- **Database Sync:** Controlled via syncDb flag in bb.js

Project Structure

```

busy-beans-backend/
├── app.js                # Express app configuration
├── bb.js                 # Server entry point
├── redis_connect.js      # Redis connection setup
├── package.json          # Dependencies and scripts
├── .env                  # Environment variables (gitignored)
├── config/               # Configuration files
│   └── config.json       # Database config (gitignored)
├── models/               # Sequelize models (37 files)
│   ├── index.js          # Model loader and associations
│   ├── user.js           # Customer/user model
│   ├── Lead.js           # Lead management model
│   ├── order.js          # Order model
│   ├── salesRep.js       # Sales representative/local partner
│   ├── product.js        # Product catalog
│   ├── supplier.js       # Supplier management
│   ├── employee.js       # Employee management
│   ├── coffeeMachine.js  # Coffee machine catalog
│   └── ... (34 more models)
├── controllers/          # Business logic
│   ├── admin/            # Admin-specific controllers (22 files)
│   │   ├── leadController.js
│   │   ├── customerController.js
│   │   ├── manageOrderController.js
│   │   ├── salesRepController.js
│   │   ├── productController.js
│   │   └── ...
│   ├── customer/         # Customer-facing controllers
│   ├── events/           # Event handlers (9 files)
│   ├── webhook/          # Webhook handlers
│   ├── errorController.js # Global error handling
│   ├── handlerFactory.js  # CRUD factory functions
│   ├── quickBooksInvoice.js
│   ├── stripe.js         # Stripe payment handling
│   └── userController.js
├── routes/               # API route definitions
│   ├── adminRoutes.js    # Admin API routes (730 lines)
│   ├── userRoutes.js     # User/customer routes
│   ├── leadRoutes.js     # Lead management routes
│   ├── qboRoutes.js      # QuickBooks integration routes
│   └── webhooks.js        # Webhook routes

```

└─ viewRoutes.js	# EJS view routes
└─ services/	# Business services (12 files)
└─ orderService.js	
└─ qboAuthService.js	
└─ qboCustomerService.js	
└─ qboInvoice.js	
└─ qboInvoiceUpdate.js	
└─ paymentSyncService.js	
└─ ...	
└─ middlewares/	# Custom middleware
└─ protect.js	# JWT authentication
└─ qboAuth.js	# QuickBooks auth
└─ qboConnected.js	# QuickBooks connection check
└─ helper/	# Email templates & helpers (28 files)
└─ transpoter.js	# Nodemailer config
└─ coffeeMachineQuotation.js	
└─ orderEmailtoCustomer.js	
└─ sentInvoiceEmail.js	
└─ userAccountCreated.js	
└─ ... (24 more email templates)	
└─ utils/	# Utility functions (20 files)
└─ appError.js	# Custom error class
└─ catchAsync.js	# Async error wrapper
└─ apiFeatures.js	# Query features (filter, sort, paginate)
└─ response.js	# Standard response format
└─ encryption.js	# Data encryption
└─ redisHandling.js	# Redis operations
└─ ...	
└─ views/	# EJS templates
└─ ... (4 view files)	
└─ public/	# Static files
└─ products/	# Product images
└─ suppliers/	# Supplier images
└─ sales-rep/	# Sales rep images
└─ machines/	# Coffee machine images



Database Models (37 Models)

Core User Models

1. ****user**** (`user.js`)

Customer/end-user accounts

- **Key Fields:** name, email, password, phoneNumber, companyName, stripeCustomerId, qboCustomerId
- **Features:** Password hashing (bcrypt), email validation, soft delete (paranoid)
- **Associations:** addresses, orders, orderFrequencies, billingAddresses, deviceTokens, userDiscounts

2. ****salesRep**** (`salesRep.js`)

Local partners/sales representatives

- **Key Fields:** srName, email, password, territoryName, partnerType (direct/dropship), connectAccountId, creditLimit
- **Features:** Stripe Connect integration, QuickBooks sync
- **Associations:** users, orders, partnerOrders, employees, addresses, qboCredentials

3. ****supplier**** (`supplier.js`)

Coffee suppliers

- **Key Fields:** name, email, address, contact info
- **Associations:** skuSuppliers, orders

4. ****employee**** (`employee.js`)

Employees under admin or sales reps

- **Key Fields:** name, email, role, permissions
- **Associations:** employeePermissions, salesRep

Product & Catalog Models

5. ****product**** (`product.js`)

Coffee products

- **Key Fields:** name, quantity, weight, unit, price, wholesalePrice, sku, grind, productCode
- **Associations:** items, partnerOrderItems, skuSuppliers, userDiscounts

6. ****category**** (`category.js`)

Product categories

- **Associations:** products

7. ****coffeeMachine**** (`coffeeMachine.js`)

Coffee machine catalog for quotations

- **Key Fields:** name, description, specifications, image

Order Management Models

8. ****order**** (`order.js`)

Customer orders

- **Key Fields:**
- Pricing: totalBill, subTotal, discountPrice, vat, shippingCharges
- Payment: paymentMethod, paymentStatus, invoiceNumber, paymentIntentId
- Status: orderStatus, trackingNumber, shippingCompany
- Frequency: frequency (just-once, weekly, every-two-weeks, every-four-weeks)
- QuickBooks: quickBooksInvoiceId, qboLastSync
- Commission: localPartnerCommission, adminReceivableAmount
- **Associations:** items, orderHistory, chequeDetail, orderFrequency, transfersToSalesRep, user, salesRep

9. ****item**** (`item.js`)

Order line items

- **Associations:** order, product

10. ****partnerOrder**** (`partnerOrder.js`)

Orders placed by local partners (wholesale)

- **Similar structure to order model**
- **Associations:** partnerOrderItems, salesRep

11. ****orderFrequency**** (`orderFrequency.js`)

Recurring order schedules

- **Key Fields:** frequency, nextOrderDate, status
- **Associations:** order, user, salesRep

12. ****orderHistory**** (`orderHistory.js`)

Order status change logs

- **Associations:** order

Lead Management Models

13. ****Lead**** (`Lead.js`)

Sales leads and enquiries

- **Key Fields:**
- Contact: company, contactName, contactPhone, contactEmail
- Address: addressLineOne, city, state, country, zipCode
- Tracking: leadSource, leadDate, status, tag (Hot/Warm/Cold)
- Pipeline: status (New Enquiry → Contacted → Quoted → Demo → Negotiation → Nurture → WON/LOST)
- Follow-up: followUpNextDate, followUpNeeded, followUpFeedback
- Quotation: quotationSent, quotationAmount, quotationDateSent
- Site Visit: siteVisitScheduled, siteVisitDate, siteVisitCompleted
- Machine Info: machineId, machineName
- Lost Lead: lostReason, customerFeedback
- **Associations:** LeadLog (activity logs)

QuickBooks Integration Models

23. ****qboCredentials**** (`qboCredentials.js`)

QuickBooks credentials

- **Associations:** salesRep

24. ****qboToken**** (`qboToken.js`)

QuickBooks OAuth tokens

- **Associations:** salesRep

25. ****qboCustomerMap**** (`qboCustomerMap.js`)

Mapping between local users and QBO customers

- **Associations:** user, salesRep



API Routes

Main Route Groups

- `/api/v1/users` - User/customer routes
- `/api/v1/admin` - Admin routes (extensive)
- `/api/v1/leads` - Lead management routes
- `/api/v1/subscription` - Subscription endpoints
- `/qbo` - QuickBooks integration routes
- `/webhook` - Webhook handlers (Stripe, etc.)
- `/view` - EJS view routes

Admin Routes Breakdown (`adminRoutes.js`)

Authentication

- `POST /api/v1/admin/login` - Admin login
- `POST /api/v1/admin/login/sales-rep` - Sales rep login
- `POST /api/v1/admin/login/supplier` - Supplier login
- `POST /api/v1/admin/forgot-password` - Password reset
- `POST /api/v1/admin/otp-verification` - OTP verification

Product Management

- `GET /api/v1/admin/product` - Get all products
- `POST /api/v1/admin/product` - Create product (with image upload)
- `GET /api/v1/admin/product/:id` - Get product by ID
- `PATCH /api/v1/admin/product/:id` - Update product
- `DELETE /api/v1/admin/product/:id` - Delete product

Order Management

- `GET /api/v1/admin/orders` - Get all orders
- `GET /api/v1/admin/order-details/:id` - Get order details
- `PATCH /api/v1/admin/order-management/update-order/:orderId` - Update order
- `DELETE /api/v1/admin/order-management/delete-order/:orderId` - Delete order
- `POST /api/v1/admin/order-management/send-invoice/:orderId` - Send invoice
- `PATCH /api/v1/admin/assign-supplier` - Assign supplier to order
- `PATCH /api/v1/admin/order-dispatch` - Mark order as dispatched
- `PATCH /api/v1/admin/order-deliver` - Mark order as delivered
- `PATCH /api/v1/admin/order-cancel` - Cancel order

Customer Management

- `GET /api/v1/admin/customer-management/customer-list/:condition` - Get customers
- `PATCH /api/v1/admin/customer-update/:id` - Update customer
- `PATCH /api/v1/admin/customer-management/assign-sale-rep/:id` - Assign sales rep
- `GET /api/v1/admin/customer-management/invoice-customers-balance` - Invoice balances
- `GET /api/v1/admin/view-customer-detail/:id` - Customer details
- `DELETE /api/v1/admin/delete-customer/:id` - Delete customer

Sales Rep Management

- `GET /api/v1/admin/sales-rep` - Get all sales reps
- `POST /api/v1/admin/sales-rep` - Create sales rep
- `GET /api/v1/admin/sales-rep/:id` - Get sales rep

- PATCH /api/v1/admin/sales-rep/:id - Update sales rep
- DELETE /api/v1/admin/sales-rep/:id - Delete sales rep

Reports

- GET /api/v1/admin/admin-reports/partner-commission - Partner commission report
- GET /api/v1/admin/admin-reports/customer-report - Customer report
- GET /api/v1/admin/admin-reports/product-sales - Product sales report
- GET /api/v1/admin/supplier-reports/assigned-orders-report/:supId - Supplier orders
- GET /api/v1/admin/sales-rep-reports/commission-summary-report/:srId - Commission summary

Dashboards

- GET /api/v1/admin/dashboard - Admin dashboard
- GET /api/v1/admin/sales-rep-dashboard/:srId - Sales rep dashboard
- GET /api/v1/admin/supplier-dashboard/:id - Supplier dashboard

Lead Routes (leadRoutes.js)

All routes require authentication and role restriction (admin, salesRep, localPartner)

- GET /api/v1/leads - Get all leads (with pagination, filtering, sorting)
- POST /api/v1/leads - Create new lead
- GET /api/v1/leads/kanban - Get leads in Kanban format (grouped by status)
- GET /api/v1/leads/stats - Get lead statistics
- GET /api/v1/leads/:id - Get lead by ID
- PATCH /api/v1/leads/:id - Update lead
- DELETE /api/v1/leads/:id - Delete lead
- POST /api/v1/leads/:id/follow-up - Schedule follow-up
- POST /api/v1/leads/:id/quotation - Send quotation
- POST /api/v1/leads/:id/site-visit - Schedule site visit
- PATCH /api/v1/leads/:id/won - Mark lead as won
- PATCH /api/v1/leads/:id/lost - Mark lead as lost

Authentication & Authorization

Middleware (middlewares/protect.js)

- **protect**: JWT token verification
- **restrictTo(...roles)**: Role-based access control

User Roles

- **admin** - Full system access
- **salesRep** / **localPartner** - Sales representative access
- **supplier** - Supplier-specific access

- **customer** - End customer access
- **employee** - Employee under admin/sales rep

Auth Flow

- Extract JWT from Authorization: Bearer header or cookies
 - Verify JWT signature
 - Check Redis for active session
 - Validate user exists and is active
 - Attach user info to req.user
-



Payment Integration

Stripe (controllers/stripe.js)

- Payment intent creation
- Card management
- Subscription handling
- Stripe Connect for sales reps
- Webhook handling

Payment Methods

- **Stripe** (immediate charge)
- **Invoice** (Net 30/60/90 terms)
- **Check** (manual reconciliation)
- **Bank Account** (ACH pullout for partners)

QuickBooks Online Integration

Services (services/)

- **qboAuthService.js**: OAuth authentication
- **qboCustomerService.js**: Customer sync
- **qboInvoice.js**: Invoice creation
- **qboInvoiceUpdate.js**: Invoice updates
- **paymentSyncService.js**: Payment synchronization

Features

- Customer sync to QBO
 - Invoice creation and sync
 - Payment recording
 - Multi-realm support (admin + partner QBO accounts)
 - Dual syncing for partner orders (admin QBO + partner QBO)
-



Email System

Email Templates (`helper/`)

28 email templates using EJS:

- **coffeeMachineQuotation.js** - Machine quotation to customer
- **coffeeMachineQuotationAdmin.js** - Machine quotation to admin
- **orderEmailtoCustomer.js** - Order confirmation
- **orderEmailtoLocalPartner.js** - Order notification to partner
- **orderDispatch.js** - Dispatch notification
- **orderShipped.js** - Shipping notification
- **sentInvoiceEmail.js** - Invoice email
- **paidInvoiceEmail.js** - Payment confirmation
- **userAccountCreated.js** - Welcome email
- **userAccountApprove.js** - Account approval
- **otpToUsers.js** - OTP for verification
- **inviteEmployee.js** - Employee invitation
- And 16 more...

Email Transporter (`helper/transpoter.js`)

Nodemailer configuration with SMTP settings

Utility Functions

Key Utilities (`utils/`)

- **apiFeatures.js**: Query filtering, sorting, pagination
- **catchAsync.js**: Async error handling wrapper
- **appError.js**: Custom error class
- **response.js**: Standardized API responses
- **encryption.js**: Data encryption/decryption
- **redisHandling.js**: Redis cache operations
- **emailsNotificationsData.js**: Email data preparation
- **throwNotification.js**: Push notification sender
- **generateInvoicePdf.js**: PDF invoice generation
- **nextFrequencyDate.js**: Recurring order date calculation
- **leadLogger.js**: Lead activity logging

Key Features

1. Lead Management System

- Complete lead pipeline (New → Contacted → Quoted → Demo → Negotiation → Nurture → WON/LOST)

- Lead tagging (Hot/Warm/Cold)
- Follow-up scheduling
- Quotation management
- Site visit scheduling
- Activity logging (LeadLog model)
- Kanban board view
- Lead statistics and analytics

2. Order Management

- Customer orders and partner orders (wholesale)
- Recurring orders (frequency-based)
- Order status tracking
- Supplier assignment
- Shipping integration
- Invoice generation and sending
- Payment tracking (Stripe + Cheque)
- QuickBooks sync

3. Multi-Tenant Support

- Admin realm
- Sales rep realms (local partners)
- Each can have their own QBO account
- Territory-based customer assignment

4. Commission System

- Automatic commission calculation for sales reps
- Partner commission tracking
- Payment pullouts from connected bank accounts
- Credit limit management

5. Discount Management

- Customer-specific discounts
- Product-level discounts
- Percentage-based pricing

6. Reporting

- Partner commission reports
- Customer reports
- Product sales reports
- Supplier order reports
- Sales rep performance reports

7. Subscription System

- Coffee machine subscriptions
- Add-on management
- Dynamic Stripe price creation

- Recurring billing

8. Recurring Orders

- Automatic order creation via Lambda function
 - Frequency-based scheduling (weekly, bi-weekly, monthly)
 - Next order date calculation
 - Customer notifications
-

Key Workflows

Complete Order Flow

```
graph TD; A[Customer Places Order] --> B[Calculate Pricing (items, discounts, shipping, VAT)]; B --> C[Payment Processing (Stripe/Invoice/Check)]; C --> D[Assign Supplier]; D --> E[Supplier Acknowledges]; E --> F[Order Dispatched (tracking added)]; F --> G[Order Delivered]; G --> H[Sync to QuickBooks]; H --> I[Commission Calculated (if sales rep involved)];
```

QuickBooks Sync Flow

```
graph TD; A[Order Created] --> B[Check QB0 Connection]; B --> C[Sync/Create Customer in QB0]; C --> D[Create Invoice in QB0]; D --> E[Store QB0 Invoice ID]; E --> F[When Payment Received → Record Payment in QB0]; F --> G[For Partner Orders → Also Sync to Partner QB0];
```

Lead Conversion Flow

```
graph TD; A[Lead Created (New Enquiry)] --> B[Contacted → Follow-ups Scheduled]; B --> C[Quotation Sent → Status: Quoted]; C --> D[Site Visit Scheduled → Status: Demo/Scheduled]; D --> E[ ];
```

Negotiation → Status: Negotiation
↓
WON → Convert to Customer → First Order

Recurring Order Automation

Lambda Trigger (Scheduled)
↓
Fetch Active Order Frequencies
↓
Check Next Order Date
↓
Create New Order
↓
Calculate Next Date
↓
Update Frequency Record
↓
Send Notification



Running the Application

Scripts (package.json)

```
npm start          # Production start (node bb.js)
npm run dev        # Development with nodemon
npm run start:prod # Production with NODE_ENV=production
npm run debug      # Debug with ndb
npm run ngrok      # Expose via ngrok
npm run dev:ngrok  # Dev + ngrok concurrently
```

Environment Variables (.env)

Required variables:

- Database credentials
 - JWT secret
 - Stripe keys
 - QuickBooks OAuth credentials
 - Email SMTP settings
 - Redis connection
-



Database Synchronization

In `bb.js`:

```
const syncDb = 0; // Set to 1 to sync database schema
```



Warning: Setting `syncDb = 1` with `{ alter: true }` will modify database schema to match models.



Security Features

- **Authentication:**
 - JWT with Redis session validation
 - Token expiry and refresh
 - Multi-device logout support
 - **Password Security:**
 - bcryptjs hashing (12 salt rounds)
 - Password reset via OTP
 - OTP expiry validation
 - **API Security:**
 - Helmet.js (HTTP headers)
 - CORS configuration
 - Rate limiting (configurable)
 - Input sanitization (express-mongo-sanitize)
 - **Data Protection:**
 - Soft deletes (paranoid models)
 - Field-level encryption utilities
 - QBO credentials encryption
-



Third-Party Integrations

Stripe

- Customer management
- Payment intents
- Subscriptions
- Connect accounts
- ACH transfers
- Webhooks

QuickBooks Online

- OAuth 2.0 authentication
- Customer sync
- Invoice creation
- Payment recording
- Multi-tenant (admin + partner realms)

Firebase

- Push notifications
- Device token management

Email (Nodemailer)

- Transactional emails

- HTML templates
 - Attachment support
-



Database Relationships

Key Associations

User → Orders:

- User hasMany Order
- User hasMany OrderFrequency
- User hasMany Address
- User hasMany BillingAddress

SalesRep → Orders:

- SalesRep hasMany User (assigned customers)
- SalesRep hasMany Order
- SalesRep hasMany PartnerOrder
- SalesRep hasMany Employee

Order → Items:

- Order hasMany Item
- Order hasMany OrderHistory
- Order hasOne OrderFrequency
- Order hasOne ChequeDetail

Lead → Assignment:

- Lead belongsTo SalesRep (optional)
- Lead belongsTo Employee (optional)
- Lead hasMany LeadLog

Subscription → Machine:

- Subscription belongsTo CoffeeMachine
 - Subscription belongsToMany Addon (through SubscriptionAddon)
-



Error Handling

- Global error handler (`errorController.js`)
 - Custom `AppError` class
 - Async error wrapper (`catchAsync`)
 - Unhandled rejection/exception handlers in `bb.js`
 - Graceful shutdown on `SIGTERM/SIGINT`
-



Performance Optimizations

- **Caching:**
 - Redis for session management
 - Token caching for auth
 - **Database:**
 - Indexes on email fields
 - Sequelize query optimization
 - Eager loading with `includes`
 - **Compression:**
 - Response compression middleware
 - Image optimization with `Sharp`
 - **Async Processing:**
 - Email sending (non-blocking)
 - Push notifications
 - QuickBooks sync (background)
-



Frontend Integration

The backend is designed to serve:

- Admin dashboard
 - Sales rep portal
 - Supplier portal
 - Customer portal
 - Mobile apps (push notifications supported)
-



Notes

- **Database:** Uses MySQL with Sequelize ORM
 - **Caching:** Redis for session and data caching
 - **File Uploads:** Multer with organized storage (products, suppliers, sales-reps, machines)
 - **Image Processing:** Sharp for image optimization
 - **PDF Generation:** Puppeteer for invoices
 - **Error Handling:** Centralized error controller with custom `AppError` class
 - **Logging:** Console-based with emojis for better visibility
-



Areas for Potential Enhancement

Based on the codebase review:

- **Testing:** No test files found - consider adding unit/integration tests
 - **Rate Limiting:** Currently commented out - enable for production
 - **API Documentation:** Consider adding Swagger/OpenAPI specs
 - **Error Logging:** Centralized error tracking (e.g., Sentry)
 - **Environment Configs:** Separate configs for dev/staging/prod
 - **Database Migrations:** Use Sequelize migrations instead of sync
 - **Code Documentation:** Add JSDoc comments to complex functions
-

Business Logic Summary

The Busy Beans Backend manages a complex B2B2C coffee distribution model:

- **Customers** place orders directly or through **Sales Reps** (local partners)
 - **Suppliers** fulfill orders
 - **Admin** oversees everything
 - **Employees** assist admins and sales reps
 - **Leads** are tracked through a sales pipeline
 - **Subscriptions** provide recurring revenue
 - **QuickBooks** maintains accounting records
 - **Stripe** handles all payments
 - **Recurring orders** automate regular deliveries
 - **Reports** provide business intelligence
-

Key Files Reference

Essential Files to Understand

- **bb.js** - Server entry point and startup
 - **app.js** - Express app configuration
 - **models/index.js** - Database model loader
 - **middlewares/protect.js** - Authentication middleware
 - **controllers/admin/manageOrderController.js** - Order management (53KB)
 - **services/qboInvoice.js** - QuickBooks invoice sync
 - **controllers/stripe.js** - Payment processing (35KB)
 - **controllers/admin/leadController.js** - Lead management (23KB)
 - **services/orderService.js** - Order service utilities
 - **services/paymentSyncService.js** - Payment sync to QBO
-

Recent Development History

Based on conversation history:

- **Lead Management Integration** (Dec 3, 2025)
 - Updated email quotation system
 - Integrated new lead data structure
 - Modified `coffeeMachineQuotation.js`
 - **Enquiry Logs Model** (Dec 3, 2025)
 - Created `enquiryLogs.js` model
 - Fields: line, status, note, description
-

Next Steps for Development

Common tasks you might work on:

- **Lead Management Enhancements**
 - Lead scoring algorithms
 - Automated follow-up reminders
 - Lead assignment rules
 - **Order Automation**
 - Automatic recurring order creation (Lambda function exists)
 - Inventory management
 - Supplier auto-assignment
 - **Reporting Enhancements**
 - Advanced analytics
 - Custom report builder
 - Export to Excel/PDF
 - **Mobile App Support**
 - Push notification improvements
 - Mobile-specific endpoints
 - Offline sync capabilities
-

End of Document

This overview provides a comprehensive understanding of the Busy Beans backend architecture, models, routes, and features. Generated on December 11, 2025.